MODULE - II

Linear Filters: Linear Filters and Convolution, Shift Invariant Linear Systems, Spatial Frequency and Fourier Transforms, Sampling and Aliasing, Filters as Templates

Edge Detection: Noise, Estimating Derivatives, Detecting Edges

Texture: Representing Texture, Analysis (and Synthesis) Using Oriented Pyramids, Application: Synthesis by Sampling Local Models, Shape from Texture.

Linear Filters and Convolution: An Overview

Linear filters and convolution are fundamental concepts in signal processing and computer vision. They are used to manipulate and analyze signals, images, and data to extract useful information or perform transformations.

Linear Filters

A **linear filter** is a system or operation that satisfies the properties of **linearity**, which include:

1. **Additivity**: The response to a sum of inputs is the sum of the responses to each input.
2. **Homogeneity**: Scaling the input scales the output by the same factor.

Linear filters are often used to:

- Remove noise.
- Enhance edges or features in an image.

- Smooth data.

Examples of Linear Filters

1. **Low-Pass Filter**: Removes high-frequency noise, often used for smoothing.
2. **High-Pass Filter**: Removes low-frequency components, emphasizing edges or details.
3. **Band-Pass Filter**: Allows a specific frequency range to pass while suppressing others.

In the context of images, linear filters are applied using **convolution.**

Linear Filters in Computer Vision

Linear filters are fundamental image processing techniques used to enhance images, detect features, or reduce noise.

These filters work by applying a **convolution operation** between an image and a filter kernel (also called a convolution mask or filter).

Mathematical Representation

A linear filter processes an image $I(x, y)$ by computing the weighted sum of pixel values in a neighborhood:

$$I'(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} w(i,j) \cdot I(x+i, y+j)$$

where:

- $I(x, y)$ is the original image.
- $I'(x, y)$ is the filtered image.
- $w(i, j)$ represents the filter kernel.

- The sum is computed over a neighborhood centered at (x, y).

Examples of Linear Filters

1. Smoothing (Blurring) Filter

Used to reduce noise by averaging the intensity of surrounding pixels.

Example: Box Blur (Mean Filter)

A simple 3×3 averaging filter:

$$w = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This replaces each pixel with the average of itself and its 8 neighbors.

2. Edge Detection Filter

Enhances edges by detecting differences in pixel intensities.

Example: Sobel Filter

Used for edge detection, applied in the x- and y-directions.

Sobel-X filter:

$$w_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel-Y filter:

$$w_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

These filters highlight vertical and horizontal edges in an image.

3. Sharpening Filter

Enhances edges and fine details by emphasizing high-frequency components.

Example: Laplacian Filter

A commonly used kernel:

$$w = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

This filter highlights regions of rapid intensity change.

Linear Filters with Convolution in Computer Vision

In computer vision, **linear filters** are widely used to process images for tasks like edge detection, smoothing, and sharpening.

A **convolution operation** is commonly used to apply these filters to an image.

Convolution Operation

Convolution is a mathematical operation that involves sliding a small matrix (called a **kernel** or **filter**) over an image and computing a weighted sum of pixel values.

Mathematically, convolution is represented as:

$$I'(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} K(i,j) \cdot I(x-i, y-j)$$

where:

- $I(x,y)$ is the original image,
- $I'(x,y)$ is the filtered image,
- $K(i,j)$ is the convolution kernel,
- k is half the size of the kernel.

Example: Applying a Linear Filter Using Convolution

Let's consider a simple example of a **3×3 averaging (blurring) filter** applied to an image.

1. Example Input Image (Grayscale, 5×5)

$$\begin{bmatrix} 10 & 20 & 30 & 40 & 50 \\ 20 & 30 & 40 & 50 & 60 \\ 30 & 40 & 50 & 60 & 70 \\ 40 & 50 & 60 & 70 & 80 \\ 50 & 60 & 70 & 80 & 90 \end{bmatrix}$$

2. 3×3 Averaging Filter (Blur Kernel)

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3. Convolution Process

- The kernel moves over the image.
- At each position, multiply corresponding values and sum them.
- The result is assigned to the center pixel of the output image.

For example, applying the kernel at position (2,2) (centered around pixel **50**):

$$\frac{1}{9} \times (30 + 40 + 50 + 40 + 50 + 60 + 50 + 60 + 70) = \frac{450}{9} = 50$$

The entire output image would be a smoothed version of the input.

Shift Invariant Linear Systems in Image Processing

A **Shift-Invariant Linear System (SILS)** is a system where the response to an input signal (such as an image) does not change if the input is shifted.

These systems are fundamental in image processing, particularly in **convolution-based filtering**.

1. Properties of a Shift-Invariant Linear System

A system is **Linear** if it follows these two principles:

1. **Additivity:** $S(f_1 + f_2) = S(f_1) + S(f_2)$
2. **Homogeneity (Scaling):** $S(af) = aS(f)$

A system is **Shift-Invariant** if shifting the input causes an identical shift in the output:

$$S(f(x - x_0, y - y_0)) = S(f)(x - x_0, y - y_0)$$

This means that applying the system to an image and then shifting it produces the same result as shifting the image first and then applying the system.

2. Example: Convolution as a Shift-Invariant Linear System

Convolution is a core operation in **linear shift-invariant (LSI) systems** in image processing. Given an input image $I(x, y)$ and a filter kernel $h(x, y)$, the output $O(x, y)$ is obtained by convolution:

$$O(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} h(i, j) I(x - i, y - j)$$

Since convolution involves a weighted sum of neighboring pixels, shifting $I(x, y)$ results in an equivalent shift in $O(x, y)$, making it **shift-invariant**.

3. Real-World Applications

1. **Blurring (Smoothing Filters)** – Reducing noise in an image.
2. **Edge Detection (Sobel, Prewitt, Laplacian Filters)** – Detecting boundaries in an image.
3. **Sharpening Filters** – Enhancing details in an image.
4. **Fourier Transform Analysis** – Frequency-based filtering using convolution in the frequency domain.

4. Example: Edge Detection using Sobel Operator

A **Sobel edge detection filter** is defined as:

$$h_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$h_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Since these filters operate through **convolution**, they are **linear and shift-invariant**. If an edge in the image shifts, the output of the Sobel filter also shifts accordingly.

4. Example: Edge Detection using Sobel Operator

A **Sobel edge detection filter** is defined as:

$$h_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$h_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Since these filters operate through **convolution**, they are **linear and shift-invariant**. If an edge in the image shifts, the output of the Sobel filter also shifts accordingly.

Spatial Frequency and Fourier Transforms in Image Processing

In image processing, **spatial frequency** and the **Fourier Transform** are essential concepts used to analyze and manipulate image details in the frequency domain.

1. Spatial Frequency

Spatial frequency refers to how image intensity changes over space (x, y coordinates). It describes the rate of intensity variation:

- **Low spatial frequencies**: Represent smooth, gradual changes (large objects, background).
- **High spatial frequencies**: Represent rapid changes (edges, fine details, noise).

Example:

- A uniform gray region has **low spatial frequency** (few intensity changes).
- A checkerboard pattern has **high spatial frequency** (many intensity changes).

2. Fourier Transform in Image Processing

The **Fourier Transform (FT)** converts an image from the **spatial domain** (x, y pixels) to the **frequency domain**, where it is represented by sinusoidal frequency components.

This helps in analyzing image details, filtering noise, and enhancing features.

Mathematical Definition (2D Fourier Transform)

The **Continuous Fourier Transform (CFT)** of an image $I(x, y)$ is given by:

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(x, y) e^{-j2\pi(ux+vy)} dx dy$$

where:

- $I(x, y)$ is the image in the spatial domain.
- $F(u, v)$ is the transformed image in the frequency domain.
- u, v are spatial frequency coordinates.

Inverse Fourier Transform (IFT)

To convert the image back to the spatial domain:

$$I(x,y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u,v) e^{j2\pi(ux+vy)} du dv$$

3. Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT)

Since images are digital, we use the **Discrete Fourier Transform (DFT)** instead of the continuous version:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x,y) e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

The **Fast Fourier Transform (FFT)** is an efficient algorithm for computing the DFT quickly.

4. Fourier Transform in Image Processing Applications

(a) Frequency Filtering

- **Low-Pass Filtering (LPF)**: Keeps low frequencies, removes high frequencies → Used for blurring/smoothing.
- **High-Pass Filtering (HPF)**: Keeps high frequencies, removes low frequencies → Used for edge detection.

(b) Image Compression

- JPEG compression uses the **Discrete Cosine Transform (DCT)** (a type of Fourier transform) to remove less important high-frequency components.

(c) Image Enhancement

- Fourier Transform helps in removing noise and enhancing features by filtering out unwanted frequency components.

5. Example: Fourier Transform of an Image

1. Convert an image to grayscale.
2. Apply **FFT** to get the frequency spectrum.
3. Shift the zero-frequency component to the center for better visualization.
4. Apply filtering (low-pass, high-pass, or band-pass).
5. Perform **Inverse FFT (IFFT)** to reconstruct the image.

Sampling and Aliasing in Image Processing

Sampling and aliasing are fundamental concepts in digital image processing that affect how images are represented and processed. Understanding these concepts is crucial for avoiding artifacts and ensuring accurate image reproduction.

1. **Sampling in Image Processing**

What is Sampling?

Sampling is the process of converting a continuous signal (such as an analog image) into a discrete signal by measuring its intensity at specific points (pixels).

In **spatial sampling**, an image is represented as a grid of pixels, with each pixel storing intensity or color information.

Sampling Rate (Resolution)

- **High sampling rate (more pixels)** → More details, better image quality.
- **Low sampling rate (fewer pixels)** → Loss of detail, blocky appearance.

Example of Sampling:

A high-resolution image with fine details can become pixelated when down sampled to a lower resolution.

2. Aliasing in Image Processing

What is Aliasing?

Aliasing occurs when a signal is undersampled (sampled at too low a rate), causing distortion or artifacts. This results in false patterns or jagged edges in images.

Why Does Aliasing Happen?

Aliasing happens due to violations of the Nyquist-Shannon Sampling Theorem, which states:

$$f_s \geq 2f_{max}$$

where:

- f_s is the sampling frequency (how often we sample),
- f_{max} is the highest frequency in the signal.

If we sample below this rate, high-frequency details become indistinguishable from lower frequencies, leading to **aliasing artifacts**.

Effects of Aliasing in Images

1. **Moiré Patterns** – Unwanted wavy or repeating patterns.
2. **Jagged Edges** (Staircase effect) – Rough edges on curves and diagonal lines.
3. **Color Artifacts** – Incorrect colors appearing in high-detail areas.

3. Preventing Aliasing

(a) Increase Sampling Rate

By increasing the resolution (more pixels per unit area), we capture more details, reducing aliasing effects.

(b) Use an Anti-Aliasing Filter (Low-Pass Filtering)

- Before sampling an image, apply a **low-pass filter (Gaussian blur)** to remove high-frequency components.
- This ensures that high-frequency details do not get misinterpreted as lower frequencies.

(c) Super-Sampling and Down-Sampling

- Render an image at a higher resolution and then downsample (resize) it smoothly.
- Commonly used in computer graphics for smoother textures.

Filters as Templates in Image Processing

In image processing, filters can act as **templates** that highlight specific features in an image by matching patterns of interest. A filter (or kernel) is a small matrix that is convolved with an image to detect structures such as edges, textures, or specific shapes.

When applying a filter to an image, we compare local regions of the image to the filter template. The stronger the match, the higher the response, making this method useful for **feature detection**.

How Filters Work as Templates

1. A filter (kernel) is designed to match a specific pattern (e.g., an edge, corner, or texture).
2. The filter slides (convolves) across the image.
3. At each position, it computes a weighted sum of pixel values.
4. The output highlights areas where the image matches the template.

3. Example: Edge Detection with a Template Filter

A **Sobel filter** is a template that detects edges in an image by emphasizing areas where pixel intensity changes rapidly.

Sobel Filter (for horizontal edges)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- This filter highlights **horizontal edges** because it detects strong intensity changes in the x-direction.

Sobel Filter (for vertical edges)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- This filter highlights **vertical edges**.

These filters act as **templates** for detecting horizontal and vertical edges.

Example: Object Detection Using a Template Filter

If we want to find a specific object (like a face or letter), we can use a small patch (template) and **match it with the image using convolution**.

- If we have an image of the letter **"A"**, we can create a filter (template) shaped like "A".
- The convolution operation will produce high values where "A" appears in the image.

Edge Detection

Edge detection is highly sensitive to noise because noise introduces small intensity variations that can be mistaken for

edges. Here's how noise affects edge detection and some common solutions:

Effects of Noise on Edge Detection:

1. **False Edges** – Random variations in pixel intensity may be detected as edges.
2. **Edge Fragmentation** – Actual edges may be broken due to noise-induced intensity fluctuations.
3. **Blurred or Weak Edges** – Noise can reduce the contrast between objects and their background, making edges less distinguishable.

Techniques to Reduce Noise Before Edge Detection:

1. **Gaussian Smoothing** – Applying a Gaussian filter before edge detection helps suppress noise while preserving edge information.
2. **Median Filtering** – A median filter is effective for removing salt-and-pepper noise while maintaining sharp edges.
3. **Bilateral Filtering** – Reduces noise while preserving edges better than Gaussian filtering.
4. **Non-Local Means (NLM) Denoising** – An advanced method that reduces noise by averaging similar patches of the image.
5. **Wavelet Denoising** – Uses wavelet transforms to separate noise from significant edge information.

Robust Edge Detection Methods for Noisy Images:

- **Canny Edge Detector** (with proper Gaussian smoothing)
- **Laplacian of Gaussian (LoG)** (detects edges after Gaussian smoothing)
- **Sobel or Prewitt Filters with Smoothing** (less sensitive to noise)
- **Edge Detection using Deep Learning (CNN-based methods)**

Estimating Derivatives for Edge Detection

Edges in an image correspond to significant changes in intensity, which can be detected by computing derivatives.

1. First-Order Derivatives (Gradient-Based Methods)

The first derivative measures the rate of intensity change. Large gradient values indicate edges.

- Gradient Magnitude:

$$\nabla I = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

- Common Operators:
  - **Sobel Operator** – Uses small convolution kernels to estimate derivatives.
  - **Prewitt Operator** – Similar to Sobel but simpler.
  - **Roberts Cross Operator** – Computes gradients at diagonals.

2. Second-Order Derivatives (Laplacian-Based Methods)

The second derivative detects intensity changes more precisely but is sensitive to noise.

- **Laplacian Operator:**

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- **Laplacian of Gaussian (LoG)** – Combines a Gaussian filter for smoothing with a Laplacian operator for edge detection.
- **Zero-Crossing Method** – Edges are detected at zero crossings of the second derivative.

3. Canny Edge Detection (Optimal Approach)

Canny uses gradient estimation with additional steps:

1. Gaussian smoothing (reduces noise).
2. Compute first derivatives using Sobel.
3. Non-maximum suppression (thin edges).
4. Hysteresis thresholding (removes weak edges).

Texture

Representing Texture in Images

Texture describes the spatial arrangement of intensity variations in an image and is crucial in pattern recognition, object detection, and image segmentation.

1. Statistical Methods (Capture texture patterns using pixel intensity distributions)

- **Gray-Level Co-occurrence Matrix (GLCM)** – Computes how often pixel intensity pairs occur at a given offset. Features include:
    - Contrast
    - Homogeneity
    - Energy
    - Correlation
- **Local Binary Patterns (LBP)** – Describes textures by thresholding a pixel's neighborhood and converting it into a binary pattern.
- **Histogram-Based Methods** – Use intensity histograms to represent texture characteristics.

2. Structural Methods (Analyze texture as repeating patterns)

- **Textons** – Fundamental texture elements, like edges, blobs, or shapes, that form a texture.
- **Wavelet Transforms** – Decompose images into different frequency bands for multi-scale texture analysis.
- **Gabor Filters** – Extract texture features by analyzing frequency and orientation components.

3. Model-Based Methods (Use mathematical models to describe textures)

Markov Random Fields (MRF) – Models texture as a probabilistic distribution of neighboring pixel relationships.

Fractal Analysis – Describes self-similar textures using fractal dimensions.

4. Transform-Based Methods (Convert images to a different domain for analysis)

Fourier Transform – Represents texture by analyzing frequency components.

Wavelet Transform – Captures multi-scale texture details.

Texture Analysis and Synthesis Using Oriented Pyramids

Oriented pyramids are multi-scale, multi-orientation image representations used for analyzing and synthesizing textures effectively. They help capture texture details at different scales and orientations.

1. Oriented Pyramids for Texture Analysis

Oriented pyramids decompose an image into multiple frequency bands and orientations, allowing detailed texture analysis.

Key Steps in Texture Analysis Using Oriented Pyramids

1. **Multi-Scale Decomposition**

Use a **Gaussian Pyramid** or **Laplacian Pyramid** to represent different levels of detail.

2. **Multi-Orientation Decomposition**

Apply oriented filters like **Gabor filters** or **Steerable Filters** to extract orientation-based texture features.

3. **Feature Extraction**

Compute energy, entropy, or statistical properties from pyramid coefficients.

4. **Texture Classification & Segmentation**

Use machine learning or deep learning models to classify texture features.

Texture Synthesis by Sampling Local Models

Synthesis by sampling local models is a technique used to generate realistic textures by learning and replicating local pixel or patch relationships from a sample texture. This method ensures that the synthesized texture maintains the statistical and structural properties of the original texture.

1. Concept of Local Models in Texture Synthesis

Instead of using global statistics, local models focus on preserving:

- **Neighborhood relationships** between pixels.
- **Patch-based structures** that represent texture patterns.
- **Statistical properties** within local regions.

The synthesis process involves sampling from these local models to generate new texture regions while maintaining consistency with the original sample.

2. Methods for Texture Synthesis Using Local Models

A. Pixel-Based Synthesis (Non-Parametric Sampling)

- **Example: Efros & Leung's Method (1999)**

    - Grows a texture pixel by pixel based on a probability distribution of similar neighborhoods.
    - Uses a similarity measure (e.g., SSD, L2 norm) to match local patches.
- **Algorithm:**
    - Start with a seed pixel or a small region.
    - Search for similar neighborhoods in the original texture.
    - Randomly select a candidate pixel based on similarity.
    - Repeat until the entire texture is synthesized.
- **Pros:**
    - Works well for highly stochastic textures.
    - Can synthesize fine details.
- **Cons:**
    - Slow and computationally expensive.
    - Sensitive to noise and errors.

B. Patch-Based Synthesis (Patch Sampling & Blending)

- **Example: Efros & Freeman's Image Quilting (2001)**
    - Copies entire patches instead of individual pixels.
    - Uses overlapping patches with optimal seam blending to avoid artifacts.
- **Algorithm:**

    - Select an initial patch from the texture sample.
    - Find the best-matching patch from the source texture.
    - Align and blend overlapping regions using minimum error boundary cut.
    - Continue until the entire texture is filled.
- **Pros:**
    - Faster than pixel-based synthesis.
    - Preserves texture structure well.
- **Cons:**
    - May produce seams or visible artifacts if blending is not optimal.

C. Multi-Resolution Synthesis (Pyramid-Based)

- Uses **Gaussian/Laplacian pyramids** or **wavelet decomposition** to synthesize textures at different scales.
- Starts synthesis at a coarse level and refines details progressively.
- Helps maintain **long-range consistency** in structured textures.

3. Applications of Local Model-Based Synthesis

- **Graphics & Gaming** – Creating seamless textures for 3D models and game environments.
- **Medical Imaging** – Synthesizing realistic textures for medical simulations.

- **Art & Design** – Generating new artistic patterns based on existing textures.
- **Super-Resolution & Inpainting** – Filling missing regions in images by sampling local models.

Shape from Texture: Understanding 3D Shape Using Texture Cues

- **Shape from texture** is a computer vision technique that estimates the 3D shape of a surface from the way textures appear in an image.
- It leverages perspective distortion, texture gradients, and foreshortening to infer depth and surface orientation.

1. **Principles of Shape from Texture**

Shape from texture relies on the following texture-based depth cues:

A. Texture Gradient

- Textures become **denser (higher frequency)** as they recede into the distance.
- Larger, more spaced-out texture elements indicate **closer regions**, while smaller, compressed ones suggest **farther regions**.

B. Foreshortening

- Textured patterns (e.g., grids, stripes) appear **stretched or compressed** depending on their surface orientation.
- A square checkerboard may look like trapezoids due to perspective distortion.

C. Perspective Distortion

- Parallel texture elements appear to **converge** as they recede, forming vanishing points.
- Linear structures (e.g., road markings) help infer depth.

D. Repetition and Regularity

- Uniform textures help estimate shape since changes in spacing and size suggest surface orientation.

2. Methods for Shape from Texture

Several computational methods analyze texture cues to recover surface shape.

A. Regular Texture Models

- Assume that textures have a known, repeating pattern.
- Changes in texture **density, orientation, or shape** indicate depth variations.

B. Statistical Texture Analysis

- **Fourier Transform** – Analyzes frequency components of textures to infer surface slant.
- **Wavelet Transforms** – Decomposes images into multi-scale features for depth estimation.

C. Geometric Methods

- **Projective Geometry Models** – Uses transformations to relate 3D surfaces to 2D projections.
- **Vanishing Point Estimation** – Detects parallel texture lines converging in perspective.

D. Machine Learning & Deep Learning

- **CNN-based Approaches** – Train deep networks to learn texture-depth relationships.
- **Self-Supervised Learning** – Uses large datasets to infer 3D shapes from single images.

3. Applications of Shape from Texture

- **3D Scene Reconstruction** – Recovering surface depth from a single image.
- **Autonomous Navigation** – Estimating terrain shape for robots and self-driving cars.
- **Medical Imaging** – Understanding surface structures in microscopy and MRI scans.
- **Augmented Reality (AR) & Virtual Reality (VR)** – Enhancing depth perception in virtual environments.